
Leant2

Answers to the Questions for Experts

A research-grounded architecture for
dependently typed program synthesis

Prepared for Vladimir Reshetnikov
21 September 2026

Review target	docs/proposals/11-further-improvements
Repository	VladimirReshetnikov/Leant2
Pinned revision	784664ff34e819422510a4d470ab07fe48bb736b
Revision date	18 September 2026, 16:22:28 UTC
Toolchain in repository	leanprover/lean4:v4.34.0
Coverage	All 31 numbered expert questions in R1–R9

Evidence status. This is a source-based research and design article, not a report of an implemented Leant2 rewrite. The mathematical claims below include their assumptions and proofs. The accompanying Python checks exercise small models of selected claims. No Lean compiler was available in the execution environment, so no Lean build, kernel replay, model-accuracy study, or Leant2 performance experiment is represented as having been run.

Abstract

Leant2’s further-improvements proposal asks how a kernel-checked synthesizer can search efficiently while inventing recursive state, exploiting examples, constructing dependent motives, retrieving components, and producing proofs. We answer all 31 questions individually. Several questions have closer published precedents than the proposal identifies: Hoogler already incorporates type classes by dictionary passing; Smyth supports higher-order examples without requiring trace completeness; Canonical demonstrates indexed motives and strengthened induction when supplied suitable induction principles; AutoLifter addresses auxiliary-state synthesis from richer specifications; and paramorphism-based synthesis provides an experimentally studied family of recursive templates. None is a complete drop-in solution for Lean’s dependent setting.

The main design recommendation is to separate five obligations: acceptance soundness, safe pruning, bounded coverage, ranking, and reproducibility. A kernel gate protects only the first unless further invariants are established. We derive sound state-capacity lower bounds from incompatible observations, a dependency-based organization of metavariable search, a conservative residual-evaluation contract, a bounded completeness theorem for top-down angelic search, and a recognizable sufficient class of locally inductive contracts. We identify invalid shortcuts involving hole-count heuristics, finite observations, carrier cardinalities, casts, and wall-clock determinism. The article concludes with an implementable architecture, staged acceptance criteria, and experiments that can settle the remaining empirical questions without overstating what existing research establishes.

Contents

1	Executive conclusions and evidence discipline	3
2	Five contracts that must not be conflated	3
3	R1: Search control and coupled goals	4
3.1	R1.1. Organizing goals that share metavariables	4
3.2	R1.2. Admissible heuristics when one assignment solves several goals	5
3.3	R1.3. Online learned costs, ranking, and finite-budget results	6
3.4	R1.4. Machine-independent behavior under a wall-clock budget	6
4	R2: Carriers, accumulators, and auxiliary state	7
4.1	R2.1. Constructive minimal carriers from finite examples	7
4.2	R2.2. Polymorphic automata and the abstract alphabet	9
4.3	R2.3. Running fusion backwards and synthesizing auxiliary functions	9
4.4	R2.4. Complete motive enumeration without the natural-number explosion	10
4.5	R2.5. Transferring induction generalization and failed-proof hints	11
5	R3: Live evaluation and conservative pruning	11
5.1	R3.1. Higher-order and dependent live bidirectional evaluation	11
5.2	R3.2. Incremental normalization under assignment and backtracking	12
5.3	R3.3. The cheapest sound fast evaluator relative to Lean	13
5.4	R3.4. Top-down angelic execution: termination and completeness	13
6	R4: Dependent motives and transport	14
6.1	R4.1. A terminating useful motive-enumeration fragment	14
6.2	R4.2. Canonical casts, heterogeneous equality, and delayed transport	15
6.3	R4.3. How much should search unfold?	16
6.4	R4.4. Reusing Lean’s motive and induction machinery	16
7	R5: Recursive capabilities and a practical scheme basis	17
7.1	R5.1. Agreement between synthesis-time recursion and Lean definitions	17
7.2	R5.2. What top-down angelic recursion gains and loses	18
7.3	R5.3. A small, measured basis of recursion schemes	18

8	R6: Universal contracts and proof debt	19
8.1	R6.1. Interleaving value holes and proof holes without an SMT oracle	19
8.2	R6.2. A recognizable class for induction, simplification, and arithmetic	20
8.3	R6.3. Automated induction in Lean as of this review	21
8.4	R6.4. A principled policy for proof debt	22
9	R7: Retrieval over a dependent, class-polymorphic library	22
9.1	R7.1. A useful over-approximation of dependent signatures	22
9.2	R7.2. Has abstraction refinement with type classes already been tried?	23
9.3	R7.3. Data goals, examples, and a program-relevance corpus	24
10	R8: Ranking and candidate equivalence	24
10.1	R8.1. A formal objective for ranking inhabitants	24
10.2	R8.2. Cheap, meaningful equivalence for dependent candidates	25
11	R9: Learned proposals behind a kernel gate	26
11.1	R9.1. Actual accuracy of model-proposed carriers and helpers	26
11.2	R9.2. Determinism and auditability beyond recording proposals	27
12	A unified implementation architecture	28
12.1	A search loop with explicit trust boundaries	28
12.2	The acceptance boundary and the search boundary	29
12.3	Where to put optimization effort	29
13	Executable model checks and what they establish	29
13.1	Minimal carriers can still overfit	29
13.2	Other checked countermodels	30
14	An experimental program for the unresolved decisions	30
14.1	Factor the benchmark dimensions	31
14.2	Measure the right quantities	31
14.3	Ablations that answer the expert questions	31
14.4	Adversarial correctness tests	31
15	Prioritized implementation plan	32
16	Conclusion	32
A	Cross-reference for all 31 expert questions	33
B	Build and reproducibility notes	34

1 Executive conclusions and evidence discipline

The proposal is strongest when it treats synthesis as a collection of fallible proposal mechanisms behind a small acceptance boundary. The next revision should extend that discipline to *negative information*: a proposal can safely be wrong, but a pruning rule cannot be wrong if the engine claims coverage of a search space. Search order, optimality, and reproducibility need separate contracts as well.

This review uses the frozen proposal, especially source files `05-search.tex` through `12-ranking.tex`; the last contains both R8 and R9. The implementation comparison is grounded in `Leant2/Engine.lean` and `Leant2/Accept/Gate.lean` at the same revision, not an assumption that every design item is already implemented. The repository’s own measurements are contextual evidence, not measurements repeated here. Its Query structure defaults to 20,000 milliseconds, 12 candidates, and a 400-millisecond grace period; the proposal’s ten-second objective is an additional evaluation target, not that structure’s current default. [1, 2, 3]

What changes the plan immediately. For retrieval, there is a direct answer to the type-class question in Hoogel+: class dictionaries can be represented as typed components in the abstract transition network. For recursion, the Para work is a more concrete starting point than an unrestricted search for arbitrary recursive bodies. For proofs, Lean already has functional induction. The adaptation work remains substantial, but these are engineering starting points rather than evidence-free possibilities. [4, 5, 6]

What requires a corrected formulation. Counting open holes is not in general an admissible lower bound on remaining search decisions. Reversal can be a plain right fold into lists, although an accumulator representation can be faster. Counting currently constructible values does not bound the semantic capacity of an infinite carrier. Agreement on probes is not a proof of function equality. A fixed hard deadline cannot both exploit arbitrarily faster machines and guarantee the same search result across them. The relevant arguments are given below, rather than asserted as empirical observations.

What remains an experiment. No reviewed source supplies the requested accuracy of current models at proposing carriers and helpers for Leant2’s exact type-and-example interface. Likewise, the best proof-debt scheduler, the practical benefit of observation-based carrier filtering, and the best ranking objective for this user population are not settled by existing benchmark numbers. Sections 11.1, 14, and 15 specify studies and release criteria for these questions.

How to interpret recommendations. A proposition with a proof is a mathematical statement about an explicitly stated model. A cited statement describes existing research or implementation. A recommended algorithm is an implementation proposal unless stated otherwise. In particular, this article does not claim a new general decision procedure for dependent synthesis, universal optimal carrier inference, or an empirically validated latency improvement.

2 Five contracts that must not be conflated

Let a frozen synthesis query be

$$Q = (\mathcal{E}, T, C, \mathcal{A}, \mathcal{G}, B),$$

where $\mathcal{E}$ is the environment, T the target type, $C : T \rightarrow \text{Prop}$ an optional contract, $\mathcal{A}$ the allowed axioms, $\mathcal{G}$ a candidate grammar, and B a collection of explicit bounds. A search state s contains

a partial term, typed metavariables, constraints, and an immutable branch identity. Write $\text{Comp}_Q(s)$ for its well-typed closing substitutions whose resulting programs lie in the declared grammar and bounds. When there is no behavioral contract, replace C by the true proposition.

Property	Required statement	What does not establish it
Acceptance	Returned $p : T$ and $\pi : C(p)$ pass the kernel and axiom policy.	Passing examples; an untrusted evaluator; a proof of a different contract.
Safe pruning	A discarded state has no completion satisfying C .	A low model score; a tactic failure; evaluator fuel exhaustion.
Bounded coverage	Every admissible candidate in $(\mathcal{G}, B)$ is considered or safely excluded.	A kernel check of the winner; top- k retrieval; a deadline.
Rank optimality	No admissible candidate has a better declared ranking value.	First success; best among candidates seen; an unrelated search-cost frontier.
Replay	A recorded query and deterministic logical schedule reproduce the claimed outcome.	Temperature zero; a random seed alone; a wall-clock limit.

These are different proof obligations. For example, silently omitting a useful provider cannot make an accepted theorem false, but it invalidates a coverage claim. Using a native evaluator to discard a branch can preserve the soundness of all returned answers while losing a valid solution. Even a complete enumeration of programs need not be complete for arbitrary propositions if the verifier is only a bounded tactic portfolio.

The negative-result algebra should therefore distinguish at least *proved impossible*, *exhausted a declared decidable finite problem*, *no certified candidate within the explored region*, and *interrupted*. A test-surviving candidate is *unrefuted*, not certified. A theorem proving impossibility is stronger than a search receipt; the latter describes an algorithmic exploration under stated assumptions.

3 R1: Search control and coupled goals

3.1 R1.1. Organizing goals that share metavariables

Answer. Use a persistent constraint hypergraph with explicit interfaces between dependent subproblems. Independent AND components can be solved and memoized separately; genuinely shared choices must remain coupled OR decisions. This improves state sharing without pretending that dependency disappears.

Aesop is an important reference for best-first proof search and configurable rule selection. Such search does not turn a dependent goal list into independent conjuncts automatically. [8]

For each open obligation g , compute a conservative footprint containing metavariables in its target, the types and values of local declarations it can use, and all recursively reachable metavariable assignments. Add universe metavariables, pending unification constraints, instance-resolution obligations, and residual contracts. Include the write footprint of permitted rules. Two goals whose printed targets contain no common metavariable may still share one through a local's type or an unresolved universe constraint.

Represent an obligation by a vertex and each shared constraint or assignable metavariable by a hyperedge. A connected component is a safe initial scheduling unit. A large component can be split at a small interface I , but then its subsearches return *relations over assignments to I* ,

not independently committed answers. Joining these relations is a constraint join; it can remain expensive. The benefit is reuse and early inconsistency detection, not removal of worst-case combinatorics.

Proposition 3.1 (A sufficient independence condition). *Suppose two subsearches share only an immutable environment and closed parameters. Their transitive read/write footprints are disjoint, new metavariables remain local, and no constraint or contract relates their results. Then successful substitutions from the two subsearches commute, up to renaming of fresh metavariables, and their solutions can be combined.*

Proof. Each substitution changes only variables outside the other's read and write footprint. It therefore preserves the other's typing judgments, constraint checks, and assignments. Applying the substitutions in either order gives the same assignments to old variables. Fresh variables can be renamed apart. The absence of a connecting obligation makes the combined constraints exactly the union of the separately satisfied constraints. $\square$

Recommended design. First implement conservative connected-component detection, not an elaborate interface solver. Persist branch states by structural sharing. Cache a successful subproblem as an abstracted proof/program template over its interface telescope, with its residual constraints. Never reuse a raw expression containing another branch's metavariable identifiers. The cache key must cover the environment, transparency and instance policy, local telescope, and interface substitution; a cache hit is replayed and type-checked. Add interface joins only after traces show substantial repeated work across small separators.

Scope of the claim. Disjointness is sufficient, not necessary. Overestimating dependencies loses optimization, not soundness. Underestimating them can corrupt search and invalidate completeness even when the final kernel gate rejects malformed survivors.

3.2 R1.2. Admissible heuristics when one assignment solves several goals

Answer. Start with zero or a maximum of verified per-obligation lower bounds. Add lower bounds only when costs have disjoint ownership. The number of open goals is not automatically admissible.

Let $d(s)$ be the minimum remaining cost of a successful completion under a fixed, nonnegative transition-cost model. An admissible heuristic satisfies $h(s) \leq d(s)$. Suppose the state contains an unknown n and many constraints $n = 0$. A single assignment $n := 0$, followed by free propagation and reflexivity closure, can discharge all constraints. Under a cost model charging the assignment as one decision, there can be arbitrarily many open obligations but remaining cost one. With a different cost model charging each distinct proof constructor, this example has a different cost; that is precisely why the cost model must be stated.

Proposition 3.2 (Safe combination of lower bounds). *If each $\ell_i(s)$ is a lower bound on the cost of every full successful completion, then $\max_i \ell_i(s)$ is admissible. If every full completion's cost decomposes into disjoint contributions c_i and $c_i \geq \ell_i(s)$, then $\sum_i \ell_i(s)$ is admissible.*

Proof. Each full completion costs at least every ℓ_i , hence at least their maximum. Under the additional disjoint decomposition, its cost is at least the sum of those bounds. Taking the infimum over completions preserves both inequalities. $\square$

A useful relaxation associates one abstract action with all obligations it could discharge together. Minimum-cost coverage, or a linear-programming relaxation of it, can provide a lower bound only if every real completion projects to an abstract cover of no greater cost. Leaving out

a real multi-goal action can make the relaxation overestimate and thus cease to be admissible. Consequently, an unverified learned estimate is an exploration priority, not an A* lower bound.

Recommended design. Charge positive cost to grammar-expanding choices and normalize genuinely administrative zero-cost steps to a canonical fixed point. Use relaxed type inhabitation for small lower bounds, combine by maximum inside coupled components, and sum only across certified independent components. Handle zero-cost cycles by canonicalization and visited-state detection. This is less aggressive than hole counting, but its stopping rule has a defensible meaning.

3.3 R1.3. Online learned costs, ranking, and finite-budget results

Answer. Separate an immutable ranking objective for completed programs from an adaptive exploration score for partial states. Learning may change which answer is discovered first; it must not silently redefine what “best” means.

Probe demonstrates the usefulness of just-in-time learning during enumerative synthesis. Its lesson is that partial successes can improve a search distribution, not that an adaptive heuristic is admissible or that wall-clock outcomes are reproducible. [9] For Leant2, define a public score $R_Q(p)$ at query initialization and a separate score $S_{Q,t}(s)$ for queue scheduling. Learned carrier probabilities, estimated proof cost, and empirical rule success belong in S , unless a versioned fixed ranking model is explicitly part of R .

Run updates at deterministic logical epochs: after a fixed number of completed expansions, aggregate observations in a canonical order, update the model, and record its digest and sufficient statistics. Do not make learning depend on which asynchronous callback happens to finish first. Keep a fair anchor queue whose service does not depend on the learned score. Adaptive guidance can then preserve eventual coverage of a fixed finite candidate set, while a deadline may interrupt that coverage.

The phrase “sound for ranking” should be replaced by a precise property. A score can be a legitimate user-facing objective without predicting preference well. It can predict preference without supporting optimal A* termination. It can also preserve accepted-answer correctness while sacrificing bounded completeness. These are separate evaluation axes.

Recommended design. Report the best *seen* certified candidate under R unless a valid frontier bound proves optimality. Evaluate learned guidance against a fixed enumeration order using the same logical work budget and the same grammar, then separately under the interactive deadline. The former isolates search quality; the latter measures practical latency. There is no reason to expect identical winners from the two comparisons.

3.4 R1.4. Machine-independent behavior under a wall-clock budget

Answer. Exact reproducibility requires a machine-independent logical schedule or replay record. Calibration can reduce variability, but cannot generally turn a deadline into an invariant amount of search.

Proposition 3.3 (Deadline trade-off). *Consider an algorithm that returns the best candidate discovered before a fixed deadline and may discover a strictly better candidate after additional positive work. Across machines with arbitrarily different execution speeds, it cannot guarantee an identical result while also exploiting that additional work whenever the deadline permits it.*

Proof. Take a run whose discoveries improve after work positions $w_1 < w_2$. Choose execution speeds so that the deadline falls between these positions on one machine and after w_2 on another. Exploiting all available work yields different best discovered candidates. To force identical

results, the faster run must suppress the improvement or both runs must use a different stopping contract. $\square$

This is not an impossibility of deterministic synthesis: a fixed logical work cap, an entirely precomputed answer, or a rule that deliberately ignores extra discoveries can be deterministic. It is a limitation of combining a hard deadline, opportunistic improvement, and hardware-invariant output.

Recommended design. Provide two internal execution contracts without requiring many user settings. The interactive path obeys a deadline and returns a certified result plus an exploration receipt. The replay path consumes the recorded logical decisions, selected proposals, and deterministic checkpoint order. For experiments, additionally support a fixed-work mode. A checkpoint contains the frontier, persistent states, model version, and ledger, not merely a candidate count.

Heartbeats or node counts alone do not make all work comparable: normalization, instance search, and proof tactics have variable cost. Use several logical counters and explicitly bound each expensive operation. A hard cancellation boundary may require an isolated worker process; a cooperative heartbeat check is not a universal wall-clock interrupt. Preserve enough budget for the final kernel check rather than returning an unchecked result when time runs out.

Anytime-algorithm research provides the right vocabulary for quality-versus-deliberation trade-offs. It does not supply a universal machine-independent schedule for this dependent search problem. [10]

4 R2: Carriers, accumulators, and auxiliary state

4.1 R2.1. Constructive minimal carriers from finite examples

Answer. There is a constructive bounded synthesis problem, but a finite grammar of *types* alone does not make it finite. Semantic state minimization, minimal type syntax, minimal program size, and efficient evaluation are different objectives.

A fold realization of $h : \text{List}(A) \rightarrow T$ consists of

$$K, \quad z : K, \quad a : A \rightarrow K \rightarrow K, \quad f : K \rightarrow T, \quad h(xs) = f(\text{foldr}(a, z, xs)).$$

The proposal bounds carrier syntax, but $K = \mathbb{N}$ still admits infinitely many states, and its step and finishing functions have infinitely many possible programs. A terminating “none in this type grammar” algorithm therefore needs further assumptions. Bound the component set, literals, term size, recursion templates, universe instantiations, and helper count. For a finite collection of closed decidable examples, an exhaustive checker over this fully specified finite space can find a minimum under a declared lexicographic cost, or exhaust that *bounded program space*. A tactic timeout must not be counted as a negative decision.

The limitation is not just the size of an enumeration. For a prescribed carrier $K = X$ and target Unit, the single empty-list observation has a realization exactly when X has a closed inhabitant: initialization supplies such an inhabitant; conversely, choose it as the initial state, retain the state at each step, and finish with the unit value. Thus a general prescribed-carrier existence procedure also addresses arbitrary inhabitation. Specialized symbolic decision procedures may exist for restricted grammars; finite type syntax alone supplies no such procedure.

There is also a clean semantic answer when the complete function is known. Define

$$xs \sim_h ys \iff \forall u \in \text{List}(A), h(u ++ xs) = h(u ++ ys).$$

This is the right-fold analogue of indistinguishability under future input. Its quotient need not be finite or have a convenient representation in the carrier grammar. Fold-characterization results and automata learning motivate this distinction; active automata learning additionally relies on queries and counterexamples rather than an arbitrary fixed passive sample. [11, 12]

Proposition 4.1 (Minimal semantic reachable state). *The quotient by $\sim_h$ admits a right-fold realization of h . Every deterministic fold realization maps onto this quotient from its reachable states.*

Proof. Set $z = []_{\sim_h}$, define $a(b, [xs]) = [b :: xs]$, and $f([xs]) = h(xs)$. The step is well defined because prepending b and then any u is another prefix. The finish is well defined by choosing the empty prefix. Induction on lists gives the claimed fold. Conversely, if two lists reach the same state of another deterministic realization, prepending the same elements preserves state equality and hence output equality. They are therefore $\sim_h$ -equivalent. Sending a reachable state to the equivalence class of any list reaching it is well defined and surjective. $\square$

Finite observations reveal only some required distinctions. Let S be the observed input–output map and V a finite set of suffixes. Construct a graph on V by adding an edge $\{x, y\}$ whenever, for some prefix u , both $u + x$ and $u + y$ are observed and their outputs differ.

Proposition 4.2 (A sound observation-based carrier bound). *If a deterministic fold realization satisfies S , its reachable states on V give a proper coloring of this graph. Thus a carrier with at most m reachable states is impossible whenever the graph has chromatic number greater than m .*

Proof. If adjacent x, y reached the same state, the common prefix witnessing their edge would drive both states through the same transitions. Their finished outputs would agree, contrary to the observations. Assigning each vertex its reached state is therefore a proper coloring. $\square$

A clique of size $m + 1$ is an easily checkable certificate that a carrier of capacity m is insufficient. General chromatic lower bounds need their own verifiable certificate. A proper m -coloring, conversely, is only a necessary-condition witness: its identifications may not extend to consistent transitions. Partial observation rows must not be declared different merely because one contains an unknown entry. Unknown observations are not inequalities.

This repairs a risky proposal in R2. The number of values the current enumerator happens to construct is *not* an upper bound on the values an eventual fold can reach. Capacity pruning is sound for a genuinely finite carrier, or a separately proved invariant restricting reachable states. It is not sound for $\mathbb{N}$, lists, or function spaces merely because a depth-limited enumeration has found few values.

A second correction concerns reversal:

$$\text{reverse}(xs) = \text{foldr}((b, r) \mapsto r + +[b], [], xs).$$

Thus reversal is a plain fold into its result type. A function carrier can avoid repeated append and improve cost, but semantic fold closure does not prove a shallow step grammar inadequate. That is a grammar-expressibility question, which must account for which providers count as one step.

Recommended design. Implement the graph lower bound for finite carriers, with certificates and an explicit reachability invariant when needed. Use partial observation tables to *rank* richer type shapes, not to claim their absence. Keep separate benchmark tracks for carrier supplied, carrier inferred within a bound, and efficient carrier inferred.

4.2 R2.2. Polymorphic automata and the abstract alphabet

Answer. The automaton view is sound for an explicitly restricted relationally parametric language. Replacing an arbitrary element type by positions or symbolic tokens is not automatically justified for all Lean programs.

For a pure polymorphic list transformer that cannot inspect elements, one can run symbolic inputs carrying distinct tokens and track which token occurrences reach the output. This captures copying, deletion, and permutation at a fixed input shape. But there are still infinitely many lengths and branching shapes. A finite set of symbolic runs is not an exact test of the whole function.

Testing-polymorphism results establish appropriate monomorphic instantiations for specified classes of polymorphic properties. They do not say that finitely many values of that instantiation determine every polymorphic function. [13] In a language with an explicit equality dictionary, programs can branch on equality patterns. If there is an order dictionary, they can inspect order relations too. The symbolic alphabet must then include the relational structure permitted by those dictionaries. Equality on positions is meaningful only when the language exposes such equality, or when it is used in the meta-level specification rather than in synthesized code.

For Lean, define a parametric search profile by construction. Whitelist constructors, eliminators, and providers equipped with suitable relationality theorems; exclude type-specific operations unavailable at the abstract type. Do not infer relational parametricity merely from a polymorphic-looking type or from passing the axiom gate. Type classes may supply computationally relevant functions, and the full environment can contain operations outside the intended parametric fragment.

Recommended design. Use a family of finite symbolic observation problems indexed by input shape and available dictionaries. For a single abstract list, tokens are element occurrences; for multiple lists, retain any specified sharing of tokens across inputs. For equality-enabled programs, enumerate realizable equality partitions rather than treating every occurrence as distinct. Cache transitions modulo renaming of tokens. This can eliminate many irrelevant concrete alphabets without claiming to eliminate unbounded structural variation.

For indexed data, symbolic observations additionally carry indices and typing witnesses. A token permutation must preserve the typing relation. Establish the renaming invariance theorem for the evaluator before using orbit representatives as a completeness-preserving reduction. Without that theorem, symbolic runs remain useful probes, not a proof that all abstract inputs have been covered.

4.3 R2.3. Running fusion backwards and synthesizing auxiliary functions

Answer. Yes, there are useful constructive precedents, especially AutoLifter, but sparse examples alone leave both the auxiliary specification and its implementation underdetermined. The most defensible complete strategy is relative to an explicitly bounded grammar.

AutoLifter studies synthesis of auxiliary functions and combinators to make an existing computation fit divide-and-conquer-like paradigms. Its input includes a richer original computation and transformation setting; it is not simply a minimal-carrier learner from a handful of examples. [14] This makes it a strong conceptual starting point for Leant2's proposed reference-implementation contracts and a weaker, proposal-generating starting point for example-only queries.

Suppose $h : \mu F \rightarrow T$ is the requested function on a polynomial inductive datatype. Search for an auxiliary $b : \mu F \rightarrow U$ and an algebra $\alpha : F(T \times U) \rightarrow T \times U$ such that

$$H = \langle h, b \rangle, \quad H \circ \text{in} = \alpha \circ F(H).$$

If this equation is proved and the datatype has its usual induction principle, H is the fold generated by α . The first projection then computes h . This is a useful proof architecture: the guessed representation and algebra have local obligations, rather than an unexplained whole-program equivalence check.

Backward search can exploit three sources of constraints. Observed output distinctions yield state-separation requirements. Failed local equations identify information absent from the current summary. A reference implementation allows counterexamples to a proposed homomorphism equation. None of these alone uniquely determines U . For example, two requested statistics often suggest a product, but the same finite observations may also be encoded by one natural number or a function closure. The representation grammar and cost objective choose among these possibilities.

Recommended design. Maintain a sequence of representation refinements. Begin with T ; after a certified obstruction or a failed proof plan, add one feature: an empty/nonempty tag, one carried input parameter, one previous value, or a second statistic. Treat the cause of a failed tactic as a hint, not a certificate that the representation is impossible. Search the finishing function and state algebra jointly, so that an invented component must contribute to the output or an invariant.

Scope of the claim. For a fixed finite grammar of typed algebras, auxiliary functions, and finishes, with decidable closed observations, exhaustive enumeration is complete for sample consistency. With universal specifications, completeness additionally requires a complete verifier for the chosen specification fragment. Polynomial functors alone do not make the synthesis or verification problem finite. There is no reviewed theorem that provides an unrestricted efficient strategy merely from that datatype assumption.

4.4 R2.4. Complete motive enumeration without the natural-number explosion

Answer. Separate structural-index generalization from representation invention, then use a stratified, fair enumeration. Completeness for an unbounded expressive language is an eventual-search property, not a promise of finite interactive success or polynomial branching.

A motive for primitive recursion with parameters can have the form

$$M(n) = S_1 \rightarrow \cdots \rightarrow S_k \rightarrow K(n),$$

where $K(n)$ may depend on the recursive index. There are two choices here: which existing parameters vary with recursive calls, and what new information the recursion returns. Enumerating both as undifferentiated arbitrary type expressions produces the combinatorial explosion described in the proposal.

Use a bounded product of grammars: a dependency-closed subset of existing parameters to generalize; a small grammar for result indices and their transports; and the carrier grammar for added state. Rank these axes separately. First try the original result motive, then the parameters that actually change in a recursive call, then one extra state constructor, and only later combinations. Deduplicate motives by a bounded canonical representation that does not identify non-definitionally-equal indices without proof.

A fairness policy can dovetail over the tuple

(generalized binders, carrier size, term size, recursion depth, helper count).

Under complete rule generation and a complete checker for the stated finite fragment, every bounded candidate is eventually reached. Expanding the bounds gives an eventual enumeration of that grammar. It gives no uniform bound on the time to a solution and does not make arbitrary Lean higher-order unification complete.

Recommended design. Keep the successful special case $T \rightarrow T$ as a high-priority instance of the grammar rather than deleting it in favor of uniform enumeration. Use evidence-guided ordering and a fair fallback. A good general mechanism can retain learned specializations without disguising them as logically necessary rules. Measure time per carrier and number of descendants, not only the count of carrier types; the expensive object is the subtree opened by a type choice.

4.5 R2.5. Transferring induction generalization and failed-proof hints

Answer. Transfer the information in failed proof plans, not an assumption that a failed tactic identifies a missing carrier. The strongest practical hints arise from variables that change in recursive calls and from invariants needed to relate carried state to the requested output.

For tail-recursive reversal, the weak statement about the empty accumulator usually does not supply the needed induction hypothesis. A useful invariant is

$$\text{revAux}(xs, acc) = \text{reverse}(xs) + +acc.$$

The accumulator must remain universally quantified in the inductive statement. In a synthesis trace, seeing that a recursive call changes acc but the available hypothesis specializes it is a precise reason to propose a function-valued motive. It is not evidence that every successful implementation must use that motive.

Similarly, a recurrence that repeatedly asks for two neighboring results suggests a paired state. A computation whose first element initializes the state suggests a sum or option-like representation rather than an arbitrary sentinel value. These are representation hypotheses obtained from a semantic mismatch, not from the surface name of a benchmark.

Canonical provides a relevant proof-side example: with an appropriate generalized induction principle available, motive synthesis can discover a strengthening needed for a theorem such as reversal involution. The distinction matters: synthesizing the motive of a supplied principle is not the same as inventing every useful principle or auxiliary lemma autonomously. [7]

Recommended design. Record failed-proof features in a compact structured form: a recursive argument differs from the argument of the available induction hypothesis; a projection used in the goal is absent from the state; a branch needs an empty-case witness; or simplification leaves a compositional equation of a known shape. Map these features to a small ranked set of carrier and invariant templates. A rippling-inspired analysis can compare the goal with the induction hypothesis and locate the mismatch, but should initially be a bounded feature extractor, not a full new theorem prover.

Keep every proposed generalization subject to ordinary type checking and proof. Preserve the original search branch while exploring the generalized one. This avoids an unsound inference from “the current proof plan failed” to “the current program or representation is impossible.”

5 R3: Live evaluation and conservative pruning

5.1 R3.1. Higher-order and dependent live bidirectional evaluation

Answer. Higher-order live example propagation is not wholly unexplored: Smyth already supports higher-order function examples. The new boundary for Leant2 is dependence, Lean’s conversion relation, and integration with mutable metavariable assignments. [15]

A residual evaluator should not represent a hole by a bare identifier. Use a typed closure

$$\langle ?m, \rho, \Gamma_m, T_m, v \rangle,$$

where ρ maps the hole's telescope into the current environment and ν identifies the immutable branch version. An application of a hole to values is a neutral term carrying that application. If the hole is later assigned a lambda, the evaluator resumes from its closure rather than substituting arbitrary syntax into a cached untyped result.

Higher-order examples can be finite application observations: a functional argument is known to return a particular result at some arguments. This is a partial specification, not an executable extensional oracle on every possible input. An unobserved application must remain unknown unless another sound mechanism evaluates it. Multiple observations of the same application must be consistent whenever equality of the inputs has been established.

Dependence creates a more fundamental issue. An expression of type $B(?n)$ cannot be treated as a value of $B(0)$ merely because one speculative branch assigned $?n := 0$. A later branch may choose $?n := 1$. Likewise, a dependent function example observes both an input x and an output in the corresponding fiber $B(x)$. Results in different fibers cannot be compared with an untyped host-language equality test.

Recommended design. Start with a pure, typed first-order observation fragment over natural numbers, booleans, lists, and products; extend it with closure-based higher-order applications; then add indexed constructors with explicit fibers and certified transports. Every extension receives a substitution-stability theorem and rollback tests. Do not initially attempt a faster replacement for every reduction rule of Lean.

Use residual expressions as constraints. For example, a known constructor mismatch in a closed output prefix can refute a branch even when another tail remains a hole, while a stuck match on an unknown index is normally just suspended. Backward propagation through a non-injective function yields a set or constraint of possible inputs, not an arbitrary chosen inverse. Losing alternatives during backward propagation is a coverage bug unless the omitted alternatives are separately explored.

5.2 R3.2. Incremental normalization under assignment and backtracking

Answer. Use a demand-driven dependency DAG with persistent identities. Ordinary memoization by expression and metavariable identifier is insufficient; rollback changes the meaning of those identifiers.

Adapton and named incremental computation provide relevant designs for dependency tracking, reuse, and from-scratch consistency. Their results do not automatically prove a Lean residual evaluator correct, but they identify the right invariant: reusing a result must agree with recomputing it in the present world. [16, 17]

Each residual node records its expression, expected type, relevant environment fingerprint, and the exact assignments on which the last result depended. A node evaluating $?x + 1$ depends on $?x$; a node that has already reduced the first branch of a known boolean match need not depend on the unused branch. Dynamic dependencies can therefore be more precise than the syntactic dependencies used for initial search independence.

A cache entry is reusable only when every dependency is unchanged. Immutable substitution-map identities, persistent content hashes, or non-reused version stamps can implement this test. A monotonically increasing counter that is rolled back and reused is not a safe identity. Neither is a global “same number of assignments” check.

Recommended design. Maintain an immutable branch store, an expression hash-consing table, a dependency index from metavariables to residual nodes, and a demand queue. On assignment, invalidate affected nodes lazily. On rollback, restore the branch-store root and recover entries whose dependency fingerprints match that root. Share closed subexpressions globally; share open expressions only through their validated closure environments. Memory accounting must

include retained old worlds, since unrestricted persistence can turn a speed optimization into a space failure.

The minimal regression is deliberately small: evaluate $?x + 1$ after assigning $?x = 0$, roll back, assign $?x = 1$, and require the result 2, not the old result 1. More demanding cases change a local instance, an index used in a result type, or an assigned metavariable reachable only through another assignment. The tests accompanying this article include the first model; the Lean-specific cases belong in the integration suite.

5.3 R3.3. The cheapest sound fast evaluator relative to Lean

Answer. The best first target is a certified evaluator for a small reified fragment, with an explicit unknown outcome outside that fragment. A complete verified clone of Lean’s evaluator is not a prerequisite.

A useful interface has three logically different outcomes:

$$\text{Value}(v, \pi), \quad \text{Refuted}(\kappa), \quad \text{Unknown}(\text{reason}).$$

Here π justifies the represented evaluation, and κ justifies rejection of the relevant candidate or partial state. The certificates need not be eagerly materialized at every intermediate node: a small verified checker can validate a compact trace or a reflected calculation when a pruning decision is committed. Cached certified summaries can amortize repeated checks.

For a partial program p , the desired pruning theorem is

$$\text{Refuted}(p, \kappa) \implies \forall \sigma \in \text{Comp}_Q(p), \neg C(p\sigma). \quad (1)$$

It is stronger than saying that the current native execution returned false. A closed wrong example can certify rejection of one completed candidate. A partial-state rejection must account for every allowed completion, often by a constructor contradiction or a formally over-approximating abstract semantics.

Lean4Lean is relevant verification infrastructure, but its publication describes an ongoing, partial verification of a Lean typechecker rather than a turnkey proof that any derived fast evaluator agrees with Lean in every case. It should be used by reusing identified definitions and established lemmas, not by transferring a blanket trust claim to a new evaluator. [18]

Recommended design. Reify a core-only datatype fragment. Prove the interpretation of each primitive operation and the substitution lemma for holes. Let the evaluator produce compact witnesses of rigid constructor disagreement, arithmetic disequality, or inconsistent observations. Require lawful equality operations: an arbitrary BEq instance is computational data, not automatically a proof of mathematical equality. Treat unsupported constants, stuck dependent transports, and resource exhaustion as unknown.

Native compiled evaluation may be a useful speculative lane and a useful test oracle. Differential testing against kernel reduction is valuable for finding bugs, but even a large generated corpus is not a soundness theorem. A faulty native refuter can silently lose the only solution while every returned answer still passes the kernel. Therefore either certify its negative results, disable its hard pruning in a coverage-preserving profile, or label that profile explicitly heuristic.

5.4 R3.4. Top-down angelic execution: termination and completeness

Answer. Angelic execution can be used top-down. The crucial invariant is that its possible behaviors include every behavior of every admissible concrete completion. Its direction of search is not itself the source of soundness or completeness.

Burst is a concrete precedent for using angelic recursive calls in bottom-up synthesis and subsequently strengthening assumptions. Smyth supplies a different route through live bidirectional constraints. Their search organizations should not be conflated, but both motivate separating observations about unfinished recursion from final execution. [19, 15]

Let $A(p, x)$ over-approximate the outputs that completions of p can produce at input x . If a required output is absent from a *proved complete* representation of $A(p, x)$, the observation refutes p . If the implementation has merely enumerated some angelic values, failure to find the required output proves nothing. This distinction is essential when the recursive result type is infinite.

A consistency table can map symbolic recursive inputs to symbolic outputs, preserving equal results for equal inputs. Allowing a fresh independent angelic output at every call gives a looser over-approximation and hence weaker pruning; it is not automatically unsound. Conversely, imposing unjustified equality or bounded-value restrictions can exclude a real completion and make pruning unsound.

Contract-relative assumptions require care. If an oracle returns the specified answer for a recursive call, that answer is justified *under the hypothesis that the candidate satisfies the contract*; it need not describe every arbitrary completion. A contradiction reached using such assumptions can refute that hypothesis, but the assumptions must be retained in the proof of the contradiction. Do not cache the resulting residual as an unconditional evaluation fact.

Proposition 5.1 (Relative completeness of bounded top-down search). *Let $\mathcal{P}$ be a finite, explicitly enumerated candidate space. Suppose search is fair, every pruning step preserves all members of $\mathcal{P}$ satisfying the observations, and closed observation checking terminates and is exact on every member. If the search runs to exhaustion, it finds a satisfying member whenever one exists.*

Proof. Fix a satisfying member. Sound pruning never removes its remaining construction path or its equivalent retained representation. Fair enumeration eventually reaches it. Exact closed checking accepts it. If no member is accepted, exhaustion and the same argument exclude a satisfying member. $\square$

The theorem does not require bottom-up version spaces. It does require that top-down branching, helper invention, and instance choices really enumerate the declared finite space. A fuel limit yielding unknown, an incomplete unifier, top- k retrieval without fallback, or a verifier restricted to unsuccessful tactic attempts prevents an unqualified exhaustion conclusion. Termination is immediate for a finite enumeration with terminating component procedures; practical usefulness is the empirical issue.

6 R4: Dependent motives and transport

6.1 R4.1. A terminating useful motive-enumeration fragment

Answer. A bounded enumeration of dependency-closed generalizations and index abstractions is both useful and terminating. It is complete for that explicitly defined syntactic fragment, not for arbitrary dependent programs or arbitrary accumulator invention.

Start with the goal telescope Γ and the proposed major premise $x : D(\vec{t})$. A generalization is not an arbitrary subset of local names: if h has a type depending on n , reverting n while leaving h fixed can be ill typed. Enumerate dependency-closed telescope suffixes or subsets, reconstructing binders in dependency order. For index expressions, enumerate non-overlapping maximal occurrences together with the dependencies needed to abstract them. “Replace every occurrence” is not a substitute for a scoping and typing check.

For example, mapping an indexed vector requires a motive schematically like

$$M(n, xs) = \text{Vec}(B, n).$$

If the result additionally carries a proof about the mapped elements, the motive must expose that dependency too. Generalizing an index appearing in a local $j : \text{Fin}(n)$ requires preserving or abstracting j appropriately. These are ordinary dependent telescope operations, not higher-order guesses with arbitrary free variables.

Canonical already gives a concrete indexed-vector example and an example of strengthened induction with a supplied generalized principle. Its documented restrictions, including limits of the implemented translation, remain relevant when comparing with full Lean. [7] Thus the right question is which bounded fragments to adapt and test, not whether any relevant dependent inhabitation technique exists.

Recommended design. Define a motive recipe as a generalized telescope, a result-type expression from a finite index grammar, an optional carrier constructor, and transport obligations. Check each recipe in a fresh transactional meta-state before opening its synthesis subtree. Keep rejection causes distinct: ill typed, outside the recipe grammar, blocked on unresolved constraints, and resource-limited. Only the first two are definitive recipe failures.

The original “maximal occurrences” idea cannot be closed under arbitrary auxiliary state: a new product component or function-valued accumulator may occur nowhere in the initial goal. Add a separate bounded carrier grammar rather than claiming that occurrence abstraction discovers it. Bounds should be named in receipts so that “all motives tried” means all recipes in that declared fragment.

6.2 R4.2. Canonical casts, heterogeneous equality, and delayed transport

Answer. Canonicalize transports relative to a small, certified index-normalization theory. Heterogeneous equality can state some relations conveniently, but does not make ill-typed applications legal or erase the need for transport.

For a family $B : I \rightarrow \text{Sort}$, an equality $e : i = j$ supports a transport

$$\text{cast}_B(e) : B(i) \rightarrow B(j).$$

Choose a terminating index normalizer N on a restricted expression language, together with proofs $e_i : i = N(i)$. Normalize boundary types to $B(N(i))$ and compose transport proofs through these canonical representatives. If two normal forms differ and the restricted theory cannot decide their equality, leave an obligation rather than inserting an unjustified coercion.

This can be effective for simple arithmetic indices and constructor forms. It is not a decision procedure for all Lean expressions. Even arithmetically true equalities can fail to be definitional; conversely, adding a propositional cast for an already definitional equality merely enlarges the search space. Test definitional equality first at a bounded transparency level.

Proof irrelevance identifies proofs of the same proposition in Lean’s conversion discipline. It does not identify arbitrary data in different fibers or make the endpoints of a cast irrelevant. A setoid quotient likewise changes the mathematical representation and needs its own congruence theorems. Neither is a universal implementation shortcut for dependent conversion.

Recommended design. Store transports as typed edges with proofs, not as untyped rewrite annotations. Collapse reflexive edges and certified compositions; prohibit zero-cost cycles in the search representation. Place casts at constructor and provider boundaries according to one normalization policy so that equivalent expressions are not generated with casts distributed at every internal node. For operations such as vector append, normalize the index equation once and reuse its proof across all corresponding terms.

Deferring a cast is safe only when the internal representation continues to track the source and destination fibers. The deferred form should be a well-typed IR node whose interpretation inserts the cast. It must not be an ordinary Lean expression that the engine temporarily treats as if its type were different.

6.3 R4.3. How much should search unfold?

Answer. Use staged, demand-driven transparency rather than universally eager unfolding or universally opaque search. The correct boundary depends on whether an operation is doing speculative retrieval, actual type checking, observation evaluation, or proof simplification.

Canonical reports preprocessing that unfolds particular aliases and definitions to expose the structures its inhabitation procedure uses. The configurable proof-search approach of `sauto` is another relevant comparison, but neither is evidence that fully unfolding every Lean definition is the best policy for this engine. [7, 20]

An alias hiding a function type may need unfolding before lambda introduction is possible. A projection hidden behind an instance may need reduction before a result type is visible. A large recursive definition, however, often deserves treatment as a component with a type and equations rather than expansion into the entire implementation. Repeatedly unfolding it can destroy indexing selectivity, increase term size, and expose representation details irrelevant to synthesis.

Recommended design. Assign a transparency policy to each phase. Retrieval uses cheap normalized heads and controlled aliases; speculative unification first uses a conservative setting; a concrete proposed application may escalate to the ordinary definitional-equality checker; evaluation unfolds only supported definitions or registered equations; proof simplification uses a bounded, explicit rewrite set. Cache results with the transparency policy in the key.

Escalation is a resumable action charged to the search ledger. Failure at a cheap transparency level is a request for more information, not proof that types are incompatible. Add per-constant unfolding budgets and explicit cycle protection for the normalization procedure. The policy should be derived from traces of the intended suite, while the kernel independently checks the final expression under its own conversion rules.

Lean’s current reference documents transparency and recursive equations, but APIs and exact behavior must be tested against the pinned toolchain. This article uses current documentation as design evidence, not as a substitute for compiling a version-specific adapter. [21, 22]

6.4 R4.4. Reusing Lean’s motive and induction machinery

Answer. Reuse Lean’s cases/induction elaboration and meta-level operations for well-specified major premises and generalizations. Wrap them as transactional proposal checkers, not as an oracle that enumerates all useful motives.

The existing induction tactic handles dependent hypotheses and explicit generalization. Functional induction follows a function’s recursive structure and is already part of Lean. [21, 6] These facilities can validate and instantiate a recipe from R4.1, produce branch telescopes, and avoid hand-reimplementing many scoping details.

A version-specific adapter should accept the major premise, the proposed generalized locals, the recursor or induction principle, and the motive recipe. It should return the instantiated eliminator, branch goals, substitutions, and generated equality obligations. Run each attempt in a saved meta-state; a failed or interrupted attempt must restore assignments, local instances, and pending constraints. Reuse only closed or correctly abstracted results.

```

tryMotive(query, branch, recipe):
  transaction = save(branch)
  check dependency-closed telescope and recipe scope
  instantiate the selected eliminator using Lean's machinery
  validate branch targets and record transport obligations
  if successful:
    return a persistent successor with its dependency interface
  otherwise:
    restore(transaction)
    return IllTyped | NotApplicable | Blocked | ResourceLimited

```

This is pseudocode, not a claimed compiled Lean API. The public adapter should hide version-sensitive details of the underlying meta operations. The low-cost path is to recognize common eliminators and reuse already instantiated telescopes, but an optimization must retain the same validation boundary.

Recommended design. Begin with lists, natural numbers, options, one indexed vector family, and a dependent pair example. Include both a successful generalized motive and a near-identical ill-scoped recipe. Measure the cost of instantiation separately from the cost of exploring the resulting branches. Only after that evidence should the engine replace validated meta-level operations with a specialized direct constructor for performance.

7 R5: Recursive capabilities and a practical scheme basis

7.1 R5.1. Agreement between synthesis-time recursion and Lean definitions

Answer. Give the synthesis language a typed semantics and prove agreement with its Lean translation. Use definitional equality where available and registered propositional equations elsewhere; neither should be assumed universally.

For a current argument x and a well-founded relation $<$, the recursive capability has the dependent form

$$rec : (y : X) \rightarrow (y < x) \rightarrow M(y).$$

The body receives no unrestricted call to itself. A synthesized recursive call must supply a smaller argument and evidence of decrease. Structural recursion is a specialized, cheap way to construct that evidence; a measure or lexicographic relation offers a broader lane. When the result depends on the input, replacing $M(y)$ by a fixed result type loses necessary typing information.

Lean's structural and well-founded elaboration provide equations with different conversion behavior. The recursive-definition reference explains the role of propositional equations for well-founded recursion; release 4.27 also introduced definitional reduction for well-founded recursion over natural-number measures in relevant cases. Hence "all well-founded equations are only propositional" would now be too strong. [22, 23]

Recommended design. Define a small typed recursive IR with a semantic interpretation. Its interpreter either produces a value together with a trace or reports unknown. For each translation scheme, establish a theorem of the form

$$\text{Eval}_k(r, x) = \text{some}(v) \implies \llbracket \text{compile}(r) \rrbracket(x) = v.$$

This is a one-way soundness statement for successful bounded evaluation. Failure to return a value at fuel k does not imply divergence or contract failure. A second, separate theorem can establish eventual success of the evaluator on an explicitly total fragment.

For direct structural schemes, prove the equation correspondence by induction on the argument. For well-founded schemes, use well-founded induction and the translation's equation

theorem. Record which equations are used in residual certificates. Do not rely on the native compiler’s operational path being the kernel’s reduction path; compilation and kernel checking are separate mechanisms. [24]

A practical first implementation can reuse validated recursor applications instead of creating named recursive declarations for every trial. Only winning candidates, or reusable helpers, need full declaration elaboration. If a named definition is required, account for its generated equations and termination proof in both the environment fingerprint and the acceptance audit. This avoids amortizing hidden elaboration work outside the stated budget.

7.2 R5.2. What top-down angelic recursion gains and loses

Answer. Top-down angelic recursion is viable. It trades bottom-up sharing of extensional summaries for stronger local type and context information. The correctness invariant is the same over-approximation condition used in R3.4; there is no general theorem that one direction must dominate.

A top-down state starts with a requested type and constraints, so it can rule out ill-typed bodies before generating many values. It can also suspend a recursive application in the exact dependent context where it occurs. Conversely, two different top-down contexts may rediscover the same useful fragment; bottom-up version spaces can share such fragments across many contexts. The recommended design is consequently hybrid: top-down typed templates with a memoized library of closed or interface-abstracted fragments.

For a fixed recursive template, allocate symbolic results to recursive calls at smaller inputs. Propagate the final examples backward through the non-recursive portions of the template, retaining all sound alternatives. Once the body is closed, execute the actual recursion. If it contradicts an assumed recursive result, refine the relevant observation constraint or reject that completed body. Do not globally reject a recursive input because one candidate gave it a different answer; counterexample caches are indexed by the specification and the assumptions that justify them.

Non-trace-complete examples are precisely where this separation is useful: the original contract need not enumerate all internal recursive calls. Nevertheless, angelic evaluation does not authorize arbitrary circular assumptions. A claimed universal recursive result needs a contract-relative argument or a later proof; it is not an unconditional fact about every unfinished candidate.

Recommended design. Combine the capability restriction of R5.1, the bounded completeness conditions of Proposition 5.1, and a final equation-based check of the realized function. Give recursive-call consistency constraints persistent identities so that they participate in the same dependency graph as ordinary holes. Test an example set omitting all internal calls, and ensure that the final recursive execution—not the angelic trace—determines whether a completed candidate satisfies the examples.

7.3 R5.3. A small, measured basis of recursion schemes

Answer. Paramorphisms are the strongest directly relevant published starting point found in this review. They expose both each original recursive substructure and its recursively computed result. The Para system uses typed multi-hole templates and searches for non-recursive fillings; its evaluation studies 59 benchmarks. Its reported setup and non-kernel validation are not Leant2’s ten-second dependent-synthesis setting. [5]

For lists, a paramorphism has the equations

$$\text{para}(z, s, []) = z, \quad \text{para}(z, s, a :: xs) = s(a, xs, \text{para}(z, s, xs)).$$

A fold is the case that ignores the original tail. Primitive recursion on natural numbers similarly exposes the predecessor and the recursively computed state. This removes the need to rediscover some common recurrence patterns while retaining more information than a plain fold.

Recommended design. Use the following layered basis, with explicit resource bounds rather than a claim that the list is uniquely canonical:

Layer	Search form and intended role
Structural	Folds and paramorphisms over registered inductive families.
Stateful	Function-valued parameters, option-like empty state, and small products.
Nested	Bounded nesting over separate arguments or substructures; includes one helper hole.
Measured	Capabilities over a small set of well-founded measures, including lexicographic ones.

This basis is a policy for allocating search effort, not a semantic classification of every task by name. Reversal can use a direct fold, a function carrier, or a helper. A library provider can hide substantial recursion. Therefore record both visible template complexity and the capabilities supplied by providers. Otherwise a benchmark can appear to demonstrate helper invention while simply retrieving the reference implementation.

Measure *coverage with an oracle template* before measuring automatic template selection. First translate each reference solution into the declared scheme grammar or exhibit why it is outside the chosen bound. Then supply the correct template and measure filling. Finally remove that hint and measure selection. These three tests distinguish missing expressiveness, weak local synthesis, and weak guidance.

A single well-founded recursion principle can be very expressive, but it is not automatically a good small search basis: the unknown relation, decreasing-call proof, and body can move all difficulty into holes. Conversely, a finite collection of primitive-recursive templates should not claim to cover every terminating Lean program. Worklist-style algorithms and sophisticated dependent recurrences need a separate measured lane or an explicit out-of-fragment result.

8 R6: Universal contracts and proof debt

8.1 R6.1. Interleaving value holes and proof holes without an SMT oracle

Answer. Use dependency-aware proof suspensions with a small, proof-producing refinement layer. Perform cheap constraints early and expensive proof search when enough of the value is stable. There is no requirement to choose between proving everything immediately and waiting until the whole program is closed.

An important implementation detail changes the interpretation of this proposal: the inspected `Engine.lean` already turns a contracted target into a subtype and extracts its value and property for the gate. The new work is therefore not merely “synthesize program and proof together.” It is to carry the specification into recursive motives so that induction hypotheses expose the properties of recursive results locally. [2]

For a length-preserving map, synthesize

$$g : (xs : \text{List}(A)) \rightarrow \{ys : \text{List}(B) \mid \text{length}(ys) = \text{length}(xs)\}.$$

In the cons case, the recursive result supplies both ys and its length equality. Constructing the new cons leaves a local successor equality. A monolithic later proof about an opaque synthesized recursor need not rediscover the same invariant.

Each proof obligation records the value holes it reads, the constraints it may assign, its stability status, and its next proof tier. A proof attempt with holes is not intrinsically meaningless: reflexivity or unification can constrain the holes, and constructor inconsistency may reject a choice immediately. What is unsafe is treating one failed tactic as evidence that all fillings are impossible. If a proof tactic assigns value metavariables, those assignments belong to the current search branch and must be rolled back on failure.

Synquid is a relevant model for decomposing specifications using refinements, but its decidable reasoning substrate is an important part of that result. [25] A Lean-only analogue can begin with a finite vocabulary of structural measures and predicates, each equipped with introduction, elimination, and monotonicity lemmas. Compute necessary constraints in this small abstract domain; attach kernel-checkable evidence to any hard rejection. Outside the domain, defer rather than guess.

Recommended design. Try inexpensive symbolic simplification as soon as a local obligation appears, bounded observation refutation once a value can be observed, and induction-aligned local proof after the branch's value choices stabilize. Escalate global proofs only for survivors. The dependency graph should wake a suspended proof when its relevant holes change; simply rotating proposition goals to the end of a list misses this structure.

8.2 R6.2. A recognizable class for induction, simplification, and arithmetic

Answer. There is a useful sufficient class: properties that are invariants of the synthesized constructor algebra and whose local verification conditions normalize into a fixed decidable theory. There is no need to recognize all true properties of recursive programs to exploit this class.

For clarity, consider lists and $g = \text{foldr}(a, z)$. Let $I(xs, r)$ describe the desired relation between the input list and its summary. The generated verification conditions are

$$I([], z), \tag{2}$$

$$\forall b, xs, r, \quad I(xs, r) \implies I(b :: xs, a(b, r)). \tag{3}$$

Proposition 8.1 (Local algebra certification). *If (2) and (3) hold, then $I(xs, g(xs))$ holds for every list xs .*

Proof. Induct on xs . The empty case is (2). In the cons case, the induction hypothesis supplies $I(xs, g(xs))$ and (3) gives $I(b :: xs, a(b, g(xs)))$, which is the desired statement by the fold equation. $\square$

The same proof applies constructor by constructor to polynomial inductive datatypes, with one induction hypothesis per recursive child. Indexed families require correctly typed motives, but the local-invariant idea remains the same.

Define a practical syntactic fragment by fixing a terminating, certified rewrite system for constructor equations and registered measures. After rewriting and applying the local induction hypotheses, require all remaining obligations to be quantifier-free linear arithmetic over natural numbers or integers, with fixed-coefficient multiplication only. Recognizing the resulting syntax is straightforward. A complete decision procedure for that fragment can then produce a proof or a genuine counterexample to the *local verification condition*. The selection of the invariant itself is still a synthesis task.

Length preservation for constructor-based map is a direct example. Sorting correctness is not automatically in this fragment: it may require permutation, membership, or order lemmas beyond arithmetic. Reversal involution normally needs an appropriate strengthening or append

lemma. Such properties become eligible only after the missing lemmas or an adequate invariant are supplied and the resulting local conditions meet the declared grammar.

Recommended design. Attach a verification class to each recursion template. Generate the local conditions mechanically; inspect their normalized syntax; dispatch only to a solver whose accepted input language contains them. Failure to enter the class means “use another verifier,” not “the program is wrong.” Even a false local condition can mean that the invariant is too strong or poorly chosen while the requested postcondition remains true; reject the particular program–invariant pair unless the counterexample also refutes the original contract.

8.3 R6.3. Automated induction in Lean as of this review

Answer. Use existing structural and functional induction as the first-line library capability. Treat automatic generalization and lemma discovery as separate services, and do not assume that adding grind or a hammer makes those services complete.

Functional induction was added before the pinned toolchain and is documented for non-mutually-recursive structural or well-founded definitions. It can follow the generated induction principle and generalize selected arguments. [6] Later try? development includes induction-oriented automation; its usefulness must be checked on the target version and task rather than inferred from the tactic’s name. [26]

Facility	Appropriate role in Leant2
induction	Apply a specified structural principle with selected generalizations.
fun_induction	Follow a named function’s recursive equations when its supported principle exists.
simp, arithmetic	Discharge normalized local conditions after the correct induction hypothesis is available.
grind	Saturate and combine supported theories inside a branch, not invent every induction invariant.
Aesop	Schedule custom proof rules, including a controlled induction rule supplied by the integration.
Canonical	Explore motives and inhabitation in its supported translation, with suitable principles available.
LeanHammer	Retrieve and combine relevant premises through its proof-search and reconstruction pipeline.

The current LeanHammer premise-selection work provides retrieval and proof-search infrastructure, not a measured solution to arbitrary induction or to Leant2’s program-plus-proof problem. [27] Similarly, this review did not locate a documented, drop-in Lean library that combines general rippling, robust lemma speculation, and predictable interactive latency for the full requested setting. That is a scoped literature-search result, not a claim that no experimental tactic exists.

Recommended design. Register a custom induction service with a small finite choice space: the program’s structural argument, its known functional induction principle, and dependency-closed generalization recipes from R4. Give each attempt a bounded simplification/arithmetic phase and a later saturation phase. Maintain separate budgets for principle selection, branch generation, and branch closing. Save the resulting proof term, not only a tactic string.

For a strict timeout, use cancellable or isolated execution where needed. Heartbeats limit selected forms of work; external solvers and native code require their own resource controls. A timed-out tactic is a suspended or failed attempt, never a proof of the negation. API availability and performance on Lean 4.34.0 still require integration tests; none were executed in this review.

8.4 R6.4. A principled policy for proof debt

Answer. Model scheduling as metareasoning about the value of additional computation, with switching costs and shared information. Do not import regret or optimality guarantees from independent bandits without checking their assumptions.

Anytime-algorithm research frames computation as a trade-off between result quality and deliberation cost; later metareasoning work also treats decisions about continuing and tuning search. [10, 28] Proof debt fits this perspective: a proof attempt may certify a candidate, refute one, generate a reusable lemma, or expose a counterexample that prunes many candidates. Its value is not simply the probability that one isolated arm succeeds.

For an action a at state s , a useful *estimated* priority is

$$V(a \mid s) = \frac{\mathbb{E}[\text{gain in certified utility or reusable information} \mid s, a]}{\mathbb{E}[\text{work}(a) \mid s] + \text{switching cost}(a)}.$$

This defines an explicit optimization target but does not make the estimate accurate. Candidate proofs share premises and counterexamples, costs change with cache state, and reward arrives only after proof checking. These dependencies violate the simplest independent-arm models.

Recommended design. Begin with deterministic staged service rather than online reinforcement learning. Each promising candidate receives a cheap local tier after passing available tests; survivors enter a bounded simplification/arithmetic tier; expensive saturation or induction gets a reserved share. Allocate a fixed minimum service to each nonempty queue at logical epochs, while using V -like heuristics to allocate the remainder. Batch obligations sharing a simplifier environment or theorem set to reduce switching cost.

Do not wait until search is “idle” to repay all expensive proof debt: an unbounded enumerator may never become idle. Similarly, a fixed exchange rate between term nodes and proof milliseconds is not portable. Measure historical costs by obligation class, but retain a deterministic accounting mode and a minimum-service guarantee.

A proof ladder should be calibrated using the repository’s actual suite; the particular millisecond values in the proposal are trial parameters, not universal constants. Evaluate total time to the first certified useful candidate, the number of expensive attempts wasted on refutable programs, and the fraction of promising candidates that time out without any proof service. The latter detects starvation that an aggregate success rate can hide.

9 R7: Retrieval over a dependent, class-polymorphic library

9.1 R7.1. A useful over-approximation of dependent signatures

Answer. Use a refinement hierarchy of typed hypergraphs. Preserve enough relationships between arguments to avoid total connectivity, but retain an explicit top element for information intentionally forgotten. Every hard exclusion needs a simulation argument from concrete applications to abstract transitions.

The first abstraction can record the result head, argument heads, sort, and function arity. The next should preserve repeated type variables: the shape $\alpha \rightarrow \alpha$ is more informative than two unrelated wildcards. For indexed families, retain small discriminating patterns such as zero versus successor, identical versus unrelated indices, and a bounded collection of arithmetic relations. Universe variables should remain symbolic constraints rather than being identified indiscriminately; Prop, Type, and universe-polymorphic families need different treatment.

A provider is a hyperedge with all its argument obligations as premises, including instance dictionaries and proof requirements. Replacing a multi-argument provider by independent

one-input edges can admit spurious paths; that is acceptable only as a consciously coarse over-approximation. It must not be mistaken for proof that the other arguments can be synthesized. Zero-argument constants, constructors, local functions, and lambda introduction also need abstract counterparts if reachability is used to preserve coverage of the full term grammar.

Formally, if α abstracts concrete typing states, require that every legal concrete provider application from s to s' is represented by an abstract transition from $\alpha(s)$ to an abstract state containing $\alpha(s')$. An abstract impossibility result is then useful. A failed concrete unification merely reveals a spurious abstract path unless it yields a genuine impossibility certificate.

Recommended design. Begin with head/arity plus shared type-variable structure and conjunctive instance guards. Refine only those index distinctions that repeatedly cause spurious applications. Keep an unfiltered fallback queue. A fixed maximum path length of three is a useful heuristic, but cannot justify excluding all longer compositions unless that restriction is part of the declared candidate grammar and the abstract path length faithfully accounts for its terms.

Classify failures by the lost information: wrong family index, incompatible repeated parameter, unsatisfied universe relation, missing dictionary, or unavailable proof argument. Use that diagnosis to refine one partition, not to reindex the entire library. Store the index by environment fingerprint and schema version, and measure preprocessing separately from per-query work.

9.2 R7.2. Has abstraction refinement with type classes already been tried?

Answer. Yes. Hoogle+ explicitly treats type classes by dictionary passing: dictionaries become ordinary typed values, instances become components producing them, and a generic function requires the appropriate dictionary as another input to its abstract transition. Its treatment is described in Sections 2.5 and 5.1. Higher-kinded class-variable cases lie beyond the specific first-order treatment presented there. [4]

For Lean, the conceptual translation is straightforward:

$$\text{generic operation} \quad \rightsquigarrow \quad (d : \text{Class}(\vec{\tau})) \rightarrow \text{ordinary arguments} \rightarrow \text{result}.$$

The engineering difficulty is that Lean's instance arguments can depend on terms, local instances, universe assignments, and other class constraints. A successful dictionary is a typed term in a particular environment. It is not merely the existence of a class name next to a type head.

Recommended design. Index providers with symbolic class preconditions. Reuse a solved dictionary only under a key containing the complete class application, relevant substitution, local-instance context, environment, and instance-resolution policy. When arguments contain unresolved metavariables, suspend the constraint or cache a conditional template; do not store a global negative result for the partially known type. For a stable closed class application, a bounded search failure still means only that the chosen instance procedure failed within its budget, unless completeness is separately established.

Do not assume coherence beyond what is actually fixed by the query environment. Two dictionaries of an operational class can induce different programs. Proof irrelevance applies to propositions, not to arbitrary structures containing comparison, addition, or monadic operations. A data-synthesis ranker may therefore need the identity and semantics of the dictionary, even when a proof-oriented symbol selector deliberately suppresses instance arguments.

The practical lesson is not to reproduce the entire Hoogle+ implementation inside Lean. It is to reuse its dictionary-as-component abstraction while retaining Lean's dependency and policy information at the boundary. Expand to higher-kinded cases only after the first-order fragment has measured selectivity and stable cache behavior.

9.3 R7.3. Data goals, examples, and a program-relevance corpus

Answer. The pinned Lean API is broad enough to submit a data goal, and it has an optional hint field that can carry an example summary. That does not imply that an existing selector understands examples or was trained for program synthesis.

In Lean 4.34.0, `Lean.LibrarySuggestions.Basic` defines the selector interface schematically as

```
Selector := MVarId -> Config -> MetaM (Array Suggestion)
-- Config includes:
-- maxSuggestions : Nat
-- caller         : Option String
-- filter         : Name -> MetaM Bool
-- hint          : Option String
```

The type does not restrict the metavariable target to `Prop`. The documentation of `hint` explicitly leaves its use to the selector. [29] Therefore a custom Leant2 selector can use a data goal plus a typed example summary through this API; a third-party selector may ignore the hint entirely.

The cloud premise-selector project describes sending goal states and new premises to a server. It also documents potentially substantial first-call costs, including 10–20 seconds of session preparation in some settings and additional server cold-start effects. Those are the project’s reported conditions, not measurements here, and already argue against making this service an unconditional inner-loop dependency of a ten-second, core-only default. [30]

Recommended design. Construct a local retrieval query from the target telescope, normalized data types, the contract’s structural symbols, and an observation sketch. Use the core interface as an adapter, but require the data-aware backend to declare that it consumes the hint. Score candidates in stages: abstract reachability, concrete typing, available instances, and only then behavioral relevance on compatible closed examples. A returned declaration name is a proposal, not evidence that its application is well typed.

Mathlib and Std definitions can supply a corpus: erase the implementation body from the input, use its accessible dependencies as initial component labels, and generate type-correct observations from the original executable definition where possible. Exclude noncomputable definitions from executable-example generation unless a suitable observational semantics is provided. Dependency labels are noisy: an implementation may use one of several interchangeable components, and inlined expressions may have no direct library label.

Prevent leakage by splitting at definition families and modules, withholding the reference definition, its aliases, its generated equations, and trivial wrappers from retrieval. Record the import environment at each training point. Evaluate recall of *some sufficient component set*, not only exact match to one reference dependency list. Follow retrieval evaluation with completed-program and proof success under equal budgets.

LeanDojo establishes useful extraction and retrieval precedents; its current project page directs new work to LeanDojo-v2. Neither the earlier theorem benchmark nor that tooling update is a measured data-goal-plus-examples corpus for this question. [31, 32]

10 R8: Ranking and candidate equivalence

10.1 R8.1. A formal objective for ranking inhabitants

Answer. A conditional Bayesian or minimum-description-length objective is defensible, but it is a declared preference model, not a theorem about user intent. It must be evaluated using

task-appropriate equivalence classes and user judgments.

Let $P_\theta(p \mid T, \mathcal{E})$ be a fixed, versioned grammar prior. Among certified candidates, one possible ranking is

$$R_Q(p) = -\log P_\theta(p \mid T, \mathcal{E}) + \lambda_1 L(p) + \lambda_2 W_Q(p) + \lambda_3 M(p),$$

where L is a chosen description cost, W_Q an explicitly defined execution-cost estimate, and M a maintainability or proof-complexity penalty. A probabilistic likelihood for imperfect examples can be added in a noisy-data mode, but ordinary Lean contracts remain hard obligations: a high likelihood is not permission to violate the supplied proposition.

There is relevant PBE ranking research. Singh and Gulwani study learning a ranking that puts an appropriate program above incorrect alternatives, rather than requiring one distinguished syntax to be highest. Their FlashFill setting provides evidence that ranking affects usability, not proof that a particular MDL objective agrees with Lean users. [33] DreamCoder provides another precedent for learned program priors and reusable libraries, again in a different setting. [34]

The present unused-input heuristic should remain a feature, not a universal first principle. A projection is supposed to ignore one component; a valid constant function may be exactly what a contract specifies. Syntactic occurrence is also not semantic influence: $x - x$ contains x but is constant on natural numbers. A data-flow over-approximation can identify some definite non-use, but it cannot generally decide which inputs can affect every program's output.

Recommended design. Publish the fixed ranking function and its stable tie-breaker. Compute reference rank modulo a specified behavioral or proved-equality class, not only by printed syntax. Include multiple acceptable references where the task is underdetermined. Gather blinded pairwise judgments about readability, expected behavior, and proof maintenance; compare these with top- k usefulness and the number of additional examples needed to distinguish candidates. A smaller term or a more input-sensitive term is not automatically the preferred answer.

The stopping rule must optimize the same objective. If the frontier lower bound applies to syntactic construction cost but the displayed ranking first penalizes unused inputs, crossing that frontier does not certify the best displayed answer. A margin-based stop can certify a corresponding cost guarantee only with an admissible lower bound for that declared cost. Otherwise report “best found” and the explored bounds.

10.2 R8.2. Cheap, meaningful equivalence for dependent candidates

Answer. Use a hierarchy: structural/definitional equality for exact deduplication, proved equality for stronger certified merging, and observational profiles for presentation and heuristic diversity. Do not label the last category as semantic equivalence.

Definitional equality is valuable because Lean can check it without inventing an extensional theorem, but normalization must be bounded in the search loop. A failure to establish equality within that bound leaves two candidates distinct; it does not prove they differ. Proof irrelevance can suppress differences in proof fields when Lean's typing and conversion rules justify it. It does not erase computational differences in arbitrary structures.

For stronger merging, a proof of $p = q$ is adequate when both terms inhabit the same type. In dependent situations, an explicitly justified transport or a suitable heterogeneous relation may be needed. Such proofs cost more than hashing and should be cached. The equality used to replace an open fragment must be a congruence for all permitted enclosing contexts, with the interface assumptions retained.

By contrast, equality on a finite probe set is not generally a congruence for program construction. The functions $f(n) = 0$ and $g(n) = n(n - 1)$ agree at 0 and 1 but disagree at 2. Replacing g by f inside a context that calls its argument at 2 changes the program. Thus probe grouping is safe

for reducing display redundancy, but unsafe as unconditional elimination of fragments from a complete search.

Recommended design. Label display groups “same outputs on these probes.” Keep their members or a recoverable generator if later contexts need them. Seek a distinguishing input by bounded symbolic testing; when found, attach the actual witness. Failure to find one is not an equality proof. Parametric testing can justify stronger reductions only inside the certified fragment discussed in R2.2.

For a dependent function $(x : A) \rightarrow B(x)$, generate x first and compare outputs using observers typed in $B(x)$. For an output containing a proof, the chosen observer may ignore that proof with a theorem justifying the projection. For a type-valued result, there may be no cheap universal equality observer at all; show structural summaries or explicit user-chosen observations rather than coercing all outputs to an untyped host value.

Retain a stable, alpha-normalized representation for exact syntactic ties. Pretty-printing depends on notation, options, and hidden arguments; identical display text is useful UI deduplication but should not serve as the general semantic identity of a candidate.

11 R9: Learned proposals behind a kernel gate

11.1 R9.1. Actual accuracy of model-proposed carriers and helpers

Answer. This review did not find a measured accuracy for the exact task: Leant2’s E8-style recursive problems, only types and examples as inputs, fixed proposal budgets, and final Lean certification. An accuracy percentage for that task would be invented. Existing neural synthesis and theorem-retrieval results motivate a study, not a numeric transfer.

DeepCoder learns program properties to guide search from examples, and DreamCoder learns reusable abstractions across tasks. LeanDojo and LeanHammer address learned guidance in theorem proving and premise retrieval. These are relevant mechanisms, but accumulator proposal, helper specification, successful completion, and successful certification are different outcomes. [35, 34, 31, 27]

Recommended design. Run an experiment with four distinct measurements. First, check whether each proposed carrier or helper signature is well formed and within the grammar. Second, measure whether a correct reference solution is expressible under at least one proposal, using an independently supplied witness when feasible. Third, run the same filler and verifier under each proposal budget. Fourth, measure end-to-end time and the fraction of certified useful answers. A model can score well on syntax and poorly on every later stage.

Compare a fixed grammar prior, a hand-ranked heuristic, an observation-graph ranker, and a model ranker. Include a carrier-given oracle condition to estimate the opportunity available to better selection. Use the same template grammar and logical completion budget for guidance-only comparisons; separately measure any expanded-grammar mode. Charge model inference, parsing, elaboration, filling, verification, and final kernel replay to the end-to-end deadline. Report cold and warm runs independently.

Avoid benchmark-name leakage. Replace descriptive names, vary irrelevant binder names, generate fresh element tokens, and hold out families rather than only different example lists of the same textbook function. Include ambiguous examples admitting simpler but unintended programs. Universal contracts or independent holdout observations are needed to distinguish sample fit from the requested behavior.

For a binary success rate $\hat{p} = x/n$, report the denominator and an interval, not only a

percentage. For example, a Wilson interval uses

$$\frac{\hat{p} + z^2/(2n) \pm z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}.$$

Repeated prompts from one task are not independent tasks; use task-level aggregation or clustered resampling. The experiment specification is a proposed protocol. No model calls or Leant2 accuracy measurements were executed for this article.

11.2 R9.2. Determinism and auditability beyond recording proposals

Answer. Separate deterministic generation, deterministic search, and deterministic proof replay. The strongest practical audit artifact is a replayable, frozen synthesis receipt whose validity does not require running the model again.

Temperature zero and a fixed seed do not by themselves specify the entire computation. Model weights, tokenizer, quantization, numerical kernels, batching, tie-breaking, and software versions can affect proposals. Reproducing a generated proposal bit for bit requires controlling these choices or recording the proposal as an input to replay. Search then needs its own logical schedule, immutable environment, and stable ordering, as discussed in R1.4.

Recommended design. Record the frozen query and contract; environment and toolchain fingerprints; grammar and bounds; original and parsed proposals; all accepted elaborated proposal types; selected declaration names and dictionary identities; logical decision order; model identifier and available generation metadata; final terms and proofs; and the transitive axiom inventory. Canonicalize ephemeral names used only in scratch environments so they do not create false replay differences. Replay the resulting program and original contract proof in a fresh process without the model.

“Proposals only reorder” has a precise consequence: every accepted proposal must pass a grammar-membership check. A model that invents a new helper or carrier outside the declared grammar has enlarged the candidate set, not merely reordered it. That can be a useful feature, but it needs a separate coverage label and an updated grammar digest. Conversely, keeping only the model’s top- k choices is not reordering unless the omitted choices remain in a fair fallback queue.

A kernel gate protects logical acceptance only relative to the statement actually checked, the axiom policy, and the checker implementation. It does not protect privacy, limit resource use, ensure the intended specification was submitted, or prevent arbitrary actions by an unrestricted elaboration script. Model output should therefore enter a restricted term/proposal interface, not an executable command file. Do not let it change imports, declare axioms, weaken the contract, alter trust options, or run arbitrary IO.

In the inspected implementation, the engine correctly forms the proof type from the frozen contract before calling the gate. The gate itself accepts an optional caller-supplied proof and proof type. A stronger future API would accept the frozen query and compute the required type $C(p)$ internally, making the caller’s present discipline a boundary invariant. This is a hardening recommendation, not a finding that the current engine proves the wrong contract. [2, 3]

The current gate’s synchronous kernel-check path and transitive axiom audit are assets worth preserving. Extend that positive acceptance boundary with independently checked pruning certificates and an honest exploration receipt; do not place the statistical model or the learned ranking function in the logical trusted base.

12 A unified implementation architecture

The nine topics should share a small set of representations rather than becoming nine independent heuristic subsystems. The central objects are a frozen query, a persistent typed search state, a declared grammar, a residual-observation store, a proof-obligation store, and a certificate ledger. The following names describe proposed components, not files claimed to exist in the repository.

Component	Responsibility	Invariant at its boundary
Query freezer	Capture target, contract, environment, axioms, grammar, and bounds.	The checked contract cannot be replaced by a proposal.
State graph	Maintain holes, dependencies, branch versions, and interfaces.	Assignments and caches belong to the correct typed world.
Proposal adapter	Supply motives, carriers, helpers, components, and guards.	A proposal is well scoped and either inside the grammar or explicitly extends it.
Residual evaluator	Reuse computations and derive observation constraints.	Hard rejection has a valid completion-preserving justification.
Recursive IR	Express allowed calls and generate equation-based realizations.	Successful evaluation agrees with the interpreted Lean term.
Proof scheduler	Service local and global obligations with bounded tactics.	Failure, timeout, refutation, and proof are distinct outcomes.
Acceptance gate	Check the final term and the original contract proof.	Kernel acceptance and axiom audit are explicit positive results.
Receipt writer	Record discoveries, exclusions, budgets, and final artifacts.	Reported coverage matches actual grammar and verification capabilities.

12.1 A search loop with explicit trust boundaries

```

synthesize(frozen_query):
    initialize persistent frontier and deterministic fallback order
    initialize observation cache, proof queues, and certificate ledger
    while a logical work unit is available and the deadline permits:
        action = choose_action_at_checkpoint()
        if action is a proposal:
            validate scope, type recipe, and grammar membership
            enqueue successors without trusting proposal relevance
        if action is residual evaluation:
            reuse only dependency-valid cache entries
            suspend unknown results
            commit a prune only with its required justification
        if action is proof work:
            run the chosen tier transactionally
            wake or refine dependent obligations
        if a program and its original contract proof are closed:
            call the acceptance gate in the frozen environment
            rank the accepted result by the fixed public objective
            preserve fair fallback service and reserve final-check work
    return certified discoveries and an accurately scoped receipt

```

This pseudocode deliberately leaves the heuristic chooser untrusted. Its choices can affect speed and which result is found first, but not the meaning of a completed certificate. Coverage

is preserved only if its fairness and pruning obligations are actually implemented. Resource interruption gives an interrupted receipt even when the grammar is finite.

12.2 The acceptance boundary and the search boundary

The inspected gate jointly generalizes remaining universe metavariables, checks declarations synchronously through the kernel path, and collects transitive axioms for comparison with the selected profile. The engine constructs contracted search goals as subtypes and supplies the corresponding property proof. [3, 2] Preserve these mechanisms while tightening the gate’s API to bind the checked proposition directly to the frozen query.

The gate should also record the exact target instantiated or specialized by a lane. A proof at a specialized universe instance is not automatically a polymorphic inhabitant of the original query. This is a general reporting requirement: a receipt must retain the actual statement the kernel checked, not only the surface query string.

The search boundary has a separate trust question. One practical configuration can require kernel-checked winners while allowing heuristic pruning, provided it claims only discovery, not exhaustive coverage. A stronger configuration can require certified pruning for the supported finite fragment. These can be internal capability levels rather than a proliferation of user settings. What matters is that their guarantees are not silently interchanged.

12.3 Where to put optimization effort

First remove repeated work with valid reuse: closed evaluation caches, instantiated provider schemas, environment indexes, and proof templates over explicit interfaces. Next reduce branching through constraints with known semantics. Only then rely on sophisticated learned priorities. A model that selects a good carrier cannot recover a branch already discarded by an unsound evaluator, and a clever scheduler cannot certify a proof that receives the wrong induction hypothesis.

Keep execution cost and term cost separate. A short provider application may run an expensive algorithm; a larger accumulator implementation may be asymptotically better. Record the intended execution model for any performance-oriented rank. Without such a model, “minimal” should mean only the declared syntactic or probabilistic objective.

13 Executable model checks and what they establish

The archive contains `model_checks.py` and its actual output, `model_checks.json`. The script uses only the Python standard library and was executed for this article. It is an independent finite model, not an implementation of Leant2, a Lean formalization, or a performance benchmark.

13.1 Minimal carriers can still overfit

Consider the unary-alphabet function

$$h(a^n) = \begin{cases} \text{true}, & n = 2, \\ \text{false}, & n \neq 2. \end{cases}$$

Initially give the synthesizer only the six observations $n = 0, \dots, 5$. The observation graph on the six suffix lengths has nine edges. Lengths 0, 1, 2, 3 form a clique, certifying that at least four

states are necessary. For each pair among these lengths, a shared observed prefix separates the outputs; for example, prefix length two separates suffix lengths zero and three.

The script exhausts all total unary Boolean-output machines with one, two, and three states, fixing initial state zero without loss of generality. The respective numbers checked are 2, 16, and 216, matching $q^q 2^q$. It then finds a four-state machine:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0, \quad \text{output true exactly at state 2.}$$

This machine satisfies all six examples and has the minimum possible number of states. Nevertheless it is wrong at length six, where it returns true. The test exposed this overfitting rather than confirming an assumption that minimum state would guarantee generalization.

Adding the counterexample $h(a^6) = \text{false}$ and rerunning the same search produces

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3, \quad \text{output true exactly at state 2.}$$

The refined machine agrees with the reference on lengths zero through 99 in the executed checks. Its general correctness also has a short mathematical proof: by induction on n , its reached state is $\min(n, 3)$; that state is two exactly when $n = 2$.

This experiment separates three claims that are often conflated: a sound lower bound on state capacity; a minimum sample-consistent realization; and identification of the intended total function. The first two held before the counterexample, while the third did not. The additional reference evaluation is an oracle supplied by this experiment, not a capability assumed for every example-only Leant2 query.

13.2 Other checked countermodels

Check	Observed finite result
Plain-fold reversal	The list-valued right fold agrees with reversal on all 2,047 binary lists of length at most ten.
Shared-goal heuristic	Four obligations can close after one charged assignment; the hole count overestimates, while the maximum individual bound is one.
Backtracking cache	A cache keyed only by the hole expression returns stale value one after the assignment changes; an assignment-aware key returns two.
Probe equality	0 and $n(n - 1)$ agree at probes zero and one, but yield zero and two at input two.
Partial observations	Different rows with no contradictory common observation do not force different states.
Retrieval conjunction	A provider requiring both A and C is unavailable from A alone, although a coarse pairwise-edge abstraction admits a spurious path.

The finite reversal check corroborates the displayed fold equation; its universal justification is the standard recursive equation for reversal. The intentionally bad cache and heuristic are tiny countermodels, not accusations that those exact bugs were observed in the repository. The distinction between model checks and engine tests is retained in the JSON scope field.

14 An experimental program for the unresolved decisions

A useful evaluation separates ability from discovery and discovery from certification. A single solved/unsolved column cannot explain whether a failure came from a missing carrier, an omitted library component, incomplete search, an incorrect candidate, or an unsuccessful proof tactic.

14.1 Factor the benchmark dimensions

For recursive tasks, cross carrier-given versus carrier-invented with template-given versus template-selected. For contracts, separate closed examples, quantified linear-measure invariants, properties needing an auxiliary lemma, and genuinely relational specifications. For retrieval, distinguish curated providers from full-library search. For dependence, include indices in results, indices in local hypotheses, nontrivial transports, and local instances affecting behavior.

Use the proposal's existing solved and stretch cases as one regression set, but add held-out families and altered specifications so the implementation cannot merely memorize those names. A claimed general carrier mechanism should not need a new hard-coded rule for each held-out family. Conversely, a well-motivated specialization should be recorded honestly rather than prohibited for ideological reasons.

14.2 Measure the right quantities

Record time to first candidate, time to first certified candidate, best fixed-rank candidate over time, total generated nodes, connected-component sizes, interface widths, normalizer work, cache-hit validity, retrieval candidate counts, instance-resolution work, proof attempts by tier, and final kernel time. Report process startup, index construction, and model inference separately and in the inclusive end-to-end total.

Run deterministic fixed-work experiments before deadline comparisons. Use identical candidate grammars and frozen environments for ablations. Repeat deadline experiments with cold and warm caches, and report distributions rather than only the fastest run. Record both sample consistency and independent correctness; a candidate that fits a few examples but fails a holdout test is not a success in an intended-program benchmark.

14.3 Ablations that answer the expert questions

Compare independent-component factoring with the same search order and no factoring. Compare the carrier graph as a ranking feature with its certified finite-capacity pruning mode. Compare residual evaluation with recomputation, verifying every cache hit against a fresh evaluation in a debug mode. Compare local invariant synthesis with a monolithic final proof, holding the program grammar fixed. Compare head-only retrieval with shared-variable and dictionary-aware retrieval. Compare fixed proof tiers with a value-of-computation scheduler under the same minimum-service policy.

For learned guidance, include the protocol of R9.1 and retain every proposal, including malformed ones and those outside the grammar. A reported proposal accuracy that excludes these failures is not the accuracy the engine experiences. Do not infer model superiority from one selected prompt or from a larger effective completion budget.

14.4 Adversarial correctness tests

A correctness suite should deliberately vary hidden dependencies: change an assignment reachable through another metavariable; restore a world and reuse an old identifier; change a local instance; use two dependent fibers whose indices are propositionally but not cheaply definitionally equal; insert a provider with a necessary second argument; and create two open fragments agreeing on probes but differing in a future context.

For every claimed exhaustive mode, run with a tiny fully enumerable grammar and compare the retained solutions against an independent exhaustive oracle. Exercise timeout paths during unification, normalization, instance search, proof construction, and final checking. No timeout

should become a logical refutation. Test the gate with a valid proof of the wrong proposition and require that the frozen-query wrapper rejects it.

15 Prioritized implementation plan

The proposal puts carrier invention first among the questions that can change the research plan. The analysis supports pursuing it, but not delaying all enabling infrastructure until a universal carrier learner is available. A useful sequence combines a small certified core with progressively better proposal mechanisms.

Stage	Deliverable	Acceptance criterion
0	Frozen-query gate and result algebra	A wrong-contract proof is rejected; unknown, exhausted, and proved-impossible outcomes remain distinct; receipt replay checks the exact statement.
1	Persistent dependency state and ledger	Assignment/rollback/instance regressions pass; tiny-grammar solution sets match an independent oracle; logical replay is stable.
2	Typed residual evaluator	Supported pruning cases have interpretation lemmas; unsupported cases return unknown; differential tests find no mismatches in the tested corpus.
3	Structural/paramorphic templates and carrier grammar	Oracle-template and oracle-carrier tracks establish coverage; finite-capacity rejections carry checked certificates; no new per-task carrier rule is needed for the declared test family.
4	Local contract motives and induction service	The linear-measure contract class is recognized and certified; failed invariants do not incorrectly refute weaker requested contracts.
5	Dependent/class-aware retrieval	Index construction is cached; local-instance changes invalidate relevant results; top- k heuristics retain an explicit fallback where coverage is claimed.
6	Ranking and optional learned proposals	Ranking is fixed per query; out-of-grammar proposals are labeled; end-to-end and fixed-work studies report completion and certification separately.

Stages can overlap, but their trust boundaries should not. In particular, a fast evaluator with no preservation argument should not quietly become the foundation of a bounded-completeness claim. Likewise, proof-locality work can proceed on supplied recursion templates before carrier invention is fully automatic, producing useful evidence about the eventual payoff of better carrier selection.

The main near-term research artifact should be a *typed lifting experiment*: one small carrier grammar, a declared recursive-template basis, certified finite-state obstructions, dependency-safe live evaluation, and local invariant generation. Test it with both oracle and automatic proposals. This concentrates effort on the actual coupling between R2, R3, R4, and R6 rather than treating each as an isolated feature.

16 Conclusion

The expert questions do not require a single unprecedented synthesis algorithm. Several have direct or near-direct research precedents; others become tractable after stating the fragment,

objective, and trust boundary precisely. The main unresolved challenge is integrating those mechanisms under a tight budget without confusing heuristic success with a mathematical guarantee.

The strongest proposed core is a persistent dependent search state, a typed residual evaluator whose negative conclusions preserve solutions, a bounded family of recursive templates with explicit carrier invention, and an induction-aligned proof service. Retrieval and learned proposals can improve discovery while remaining outside the acceptance trusted base. Ranking and receipts should say exactly what was optimized and explored.

The four-state experiment gives a compact summary of the overall lesson. A valid capacity proof and a minimum sample-consistent program can coexist with the wrong behavior. The engine needs both good proposals and a stronger route to the intended specification. Kernel certification, counterexample refinement, and honest result categories are complementary parts of that route, not interchangeable substitutes.

A Cross-reference for all 31 expert questions

The identifiers below follow the order of each original “Questions for experts” list. Question descriptions are paraphrases; every answer is in the indicated section.

ID	Question focus	Main answer category	Page
R1.1	Organization of coupled metavariable goals	Dependency interfaces and persistence	4
R1.2	Admissible shared-goal heuristic	Maximum bounds; conditional sums	5
R1.3	Online costs and ranking controls	Fixed objective, adaptive exploration	6
R1.4	Machine-independent deadlines	Logical replay and explicit trade-off	6
R2.1	Minimal carriers from finite examples	Bounded synthesis and capacity proofs	7
R2.2	Polymorphic automata and alphabets	Certified parametric fragments	9
R2.3	Backward fusion and auxiliary invention	Typed lifting with stronger oracles	9
R2.4	Complete practical motive enumeration	Separate bounded grammars	10
R2.5	Generalization and failed-proof hints	Invariant-guided proposals	11
R3.1	Higher-order/dependent live evaluation	Typed closures and fibers	11
R3.2	Incremental normalization and rollback	Persistent dependency-valid caching	12
R3.3	Fast conservative evaluator	Small certified fragment	13
R3.4	Top-down angelic search guarantees	Over-approximation and bounded theorem	13
R4.1	Useful complete motive fragment	Dependency-closed recipes	14
R4.2	Canonical casts and equality	Certified index normalization	15
R4.3	Transparency and unfolding	Staged policies and fallbacks	16

ID	Question focus	Main answer category	Page
R4.4	Lean motive APIs	Transactional versioned adapters	16
R5.1	Recursive evaluator/Lean agreement	Typed IR and equation proofs	17
R5.2	Top-down recursive angelic search	Hybrid sharing with final realization	18
R5.3	A measured small recursion basis	Paramorphisms and layered extensions	18
R6.1	Value/proof interleaving	Local refinements and suspensions	19
R6.2	Recognizable inductive contract class	Constructor-algebra verification	20
R6.3	Current automated induction in Lean	Existing induction plus bounded services	21
R6.4	Principled proof-debt scheduling	Metareasoning with service guarantees	22
R7.1	Dependent signature abstraction	Refined typed hypergraphs	22
R7.2	Type classes in abstraction refinement	Existing dictionary-passing precedent	23
R7.3	Data/example retrieval and training corpus	Actual API support; new data adapter	24
R8.1	Ranking objective and preference	Fixed MDL/Bayesian model and study	24
R8.2	Dependent candidate equivalence	Certified equality versus probe groups	25
R9.1	Model accuracy for carriers/helpers	No transferred percentage; study protocol	26
R9.2	Determinism and auditability	Frozen receipts and independent replay	27

B Build and reproducibility notes

The source is a self-contained `article.tex`, including its bibliography. Compile with `latexmk-pdflatexarticle.tex`, or run `pdflatexarticle.tex` repeatedly until references stabilize. Standard TeX Live packages are used, including New PX fonts, `microtype`, and `hyperref`. No network access is required to rebuild once those packages are installed.

Run `python3model_checks.py` to regenerate the JSON results. The checks enumerate small finite models and do not need Lean, Mathlib, an external solver, or a model service. Their output contains no timing claim and no hardware-dependent random sampling. The source includes explicit validation rather than assertions that can be disabled with Python’s optimization flag.

The research used repository source and primary publications or official documentation. Web and repository resources were consulted on 21 September 2026. Live documentation and auxiliary project READMEs can change; the principal Leant2 source and the library-suggestion API are pinned in their references. No attempt is made to redistribute the reviewed repositories or third-party papers in this archive.

References

- [1] V. Reshetnikov. *Leant2: Further improvements*, especially sections R1–R9 and the summary of outside-expertise questions. Repository revision 784664ff34e819422510a4d470ab07fe48bb736b, 18 September 2026. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements>.
- [2] V. Reshetnikov. *Leant2/Engine.lean*, same revision. In particular: `Query`, `candidateCost`, `rankCandidates`, and `runQuery`. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Engine.lean>.
- [3] V. Reshetnikov. *Leant2/Accept/Gate.lean*, same revision. In particular: `generalizeLevels`, `kernelCheckAndAudit`, and `gate`. <https://github.com/VladimirReshetnikov/Leant2/blob/784664ff34e819422510a4d470ab07fe48bb736b/Leant2/Accept/Gate.lean>.
- [4] Z. Guo, M. James, D. Justo, J. Zhou, Z. Wang, R. Jhala, and N. Polikarpova. Program synthesis by type-guided abstraction refinement. *Proceedings of the ACM on Programming Languages*, POPL, 2020. <https://doi.org/10.1145/3371080>; <https://arxiv.org/abs/1911.04091>.
- [5] Q. Hong and A. Aiken. Recursive program synthesis using paramorphisms. *Proceedings of the ACM on Programming Languages* 8 (PLDI), article 151, pp. 102–125, 2024. <https://doi.org/10.1145/3656381>.
- [6] Lean developers. *Lean 4.18.0 release notes*, functional-induction support, 2025. <https://lean-lang.org/doc/reference/latest/releases/v4.18.0/>.
- [7] C. Norman and J. Avigad. Canonical for automated theorem proving in Lean. *Interactive Theorem Proving*, LIPIcs, article 14, 2025. <https://doi.org/10.4230/LIPIcs.ITP.2025.14>; <https://arxiv.org/html/2504.06239v2>.
- [8] J. Limperg and A. H. From. Aesop: White-box best-first proof search for Lean. *Certified Programs and Proofs*, pp. 253–266, 2023. <https://doi.org/10.1145/3573105.3575671>.
- [9] S. Barke, H. Peleg, and N. Polikarpova. Just-in-time learning for bottom-up enumerative synthesis. *Proceedings of the ACM on Programming Languages*, OOPSLA, 2020. <https://doi.org/10.1145/3428295>; <https://arxiv.org/abs/2010.08663>.
- [10] S. Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine* 17(3), pp. 73–83, 1996. <https://doi.org/10.1609/aimag.v17i3.1232>.
- [11] J. Gibbons, G. Hutton, and T. Altenkirch. When is a function a fold or an unfold? *Electronic Notes in Theoretical Computer Science* 44(1), 2001. [https://doi.org/10.1016/S1571-0661\(04\)80906-X](https://doi.org/10.1016/S1571-0661(04)80906-X).
- [12] D. Angluin. Learning regular sets from queries and counterexamples. *Information and Computation* 75(2), pp. 87–106, 1987. [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6).
- [13] J.-P. Bernardy, P. Jansson, and K. Claessen. Testing polymorphic properties. *European Symposium on Programming*, 2010. https://doi.org/10.1007/978-3-642-11957-6_8; <https://research.chalmers.se/en/publication/99387>.
- [14] R. Ji, Y. Zhao, Y. Xiong, D. Wang, L. Zhang, and Z. Hu. Decomposition-based synthesis for applying divide-and-conquer-like algorithmic paradigms. *arXiv:2202.12193*, version 3. <https://arxiv.org/html/2202.12193v3>.
- [15] J. Lubin, N. Collins, C. Omar, and R. Chugh. Program sketching with live bidirectional evaluation. *Proceedings of the ACM on Programming Languages*, ICFP, 2020. <https://doi.org/10.1145/3408991>; <https://arxiv.org/html/1911.00583v2>.
- [16] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster. Adapton: Composable, demand-driven incremental computation. *Programming Language Design and Implementation*, 2014. <https://doi.org/10.1145/2594291.2594324>; <https://adapton.org/>.
- [17] M. A. Hammer et al. Incremental computation with names. *Object-Oriented Programming, Systems, Languages, and Applications*, 2015. <https://doi.org/10.1145/2814270.2814305>; <https://arxiv.org/abs/1503.07792>.
- [18] M. Carneiro. Lean4Lean: Verifying a typechecker for Lean, in Lean. *Types for Proofs and Programs (TYPES 2025)*, LIPIcs 384, article 2, published 2026. <https://doi.org/10.4230/LIPIcs.TYPES.2025.2>; <https://research.chalmers.se/publication/553480>.
- [19] A. Miltner et al. Bottom-up synthesis of recursive functional programs using angelic execution. *Proceedings of the ACM on Programming Languages*, POPL, 2022. <https://arxiv.org/abs/2107.06253>.

- [20] Ł. Czajka. Practical proof search for Coq by type inhabitation. *International Joint Conference on Automated Reasoning*, 2020. https://doi.org/10.1007/978-3-030-51054-1_3. See also the authors' documentation: <https://coqhammer.github.io/>.
- [21] Lean developers. *Lean Language Reference: Tactic Reference*. Live reference consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [22] Lean developers. *Lean Language Reference: Recursive Definitions*. Live reference consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.
- [23] Lean developers. *Lean 4.27.0 release notes*, reduction of natural-number-measure well-founded recursion, 2026. <https://lean-lang.org/doc/reference/latest/releases/v4.27.0/>.
- [24] Lean developers. *Lean Language Reference: Elaboration and Compilation*. Live reference consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/>.
- [25] N. Polikarpova, I. Kuraj, and A. Solar-Lezama. Program synthesis from polymorphic refinement types. *Programming Language Design and Implementation*, 2016. <https://doi.org/10.1145/2908080.2908093>; <https://arxiv.org/abs/1510.08419>.
- [26] Lean developers. *Lean 4.26.0 release notes*, induction-oriented try? development, 2025. <https://lean-lang.org/doc/reference/latest/releases/v4.26.0/>.
- [27] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck. Premise selection for a Lean hammer. *arXiv:2506.07477v2*, 2026; first version 2025. <https://arxiv.org/html/2506.07477v2>.
- [28] A. Bhatia, J. Svegliato, S. B. Nashed, and S. Zilberstein. Tuning the hyperparameters of anytime planning: A metareasoning approach with deep reinforcement learning. *International Conference on Automated Planning and Scheduling*, 2022. <https://doi.org/10.1609/icaps.v32i1.19842>.
- [29] Lean developers, K. Morrison et al. *Lean/LibrarySuggestions/Basic.lean*, Lean 4.34.0, especially Config and Selector. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/LibrarySuggestions/Basic.lean>.
- [30] LeanHammer premise-selection developers. *premise-selection README*, interface, privacy-relevant data flow, and runtime/cache notes. Consulted 21 September 2026. <https://github.com/hanwenzhu/premise-selection/blob/main/README.md>.
- [31] K. Yang et al. LeanDojo: Theorem proving with retrieval-augmented language models. *Neural Information Processing Systems*, 2023. <https://arxiv.org/abs/2306.15626>.
- [32] LeanDojo project. *LeanDojo-v2: A comprehensive library for AI-assisted theorem proving in Lean*. Project page and migration notice, consulted 21 September 2026. <https://leandojo.org/leandojo>.
- [33] R. Singh and S. Gulwani. Predicting a correct program in programming by example. *Computer Aided Verification*, 2015. Authors' publication page: <https://www.microsoft.com/en-us/research/publication/predicting-a-correct-program-in-programming-by-example/>.
- [34] K. Ellis et al. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. *Programming Language Design and Implementation*, 2021. Related authors' preprint, *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning*: <https://arxiv.org/abs/2006.08381>.
- [35] M. Balog, A. L. Gaunt, M. Brockschmidt, S. Nowozin, and D. Tarlow. DeepCoder: Learning to write programs. *International Conference on Learning Representations*, 2017. <https://arxiv.org/abs/1611.01989>.